SysML Cloud Platform

2025-12-03

Contents

System Engineering Management Plan (SEMP)

SysML Cloud Platform

Document Version: 1.0
Last Updated: 2025-12-03
Prepared by: Systems Engineering Team
Status: Active

---

1. Purpose and Scope

The **System Engineering Management Plan (SEMP)** defines how systems engineering will be planned, organized, conducted, and controlled for the **SysML Cloud Platform** project. This plan ensures that all systems engineering activities align with project objectives and organizational standards.

1.1 Project Vision

Transform system models into actionable implementation artifacts through a Git-native, model-driven engineering framework that connects SysML v2 system architecture with cloud infrastructure, technicians, and developers.

1.2 Scope

- Model-driven design of datacenter and cloud infrastructure
- Integration of SysML requirements, architecture, and deployment models
- Automated generation of documentation, diagrams, and deployment artifacts
- Bidirectional traceability from requirements through implementation and operations

---

2. Organizational Structure

2.1 SE Organization

```
Systems Engineering Manager
  Architecture Lead (SysML Modeling)
  Infrastructure Engineer (Deployment & Validation)
  Integration Engineer (CI/CD & Tooling)
  Documentation & Quality Lead
```

2.2 Roles and Responsibilities

Role	Responsibilities
SE Manager	Oversees SE processes, stakeholder coordination, risk management, and metrics
Architecture Lead	Develops and maintains SysML models; defines system structure and behavior
Infrastructure Engineer	Translates SysML models into deployment infrastructure; performs validation
Integration Engineer	Maintains CI/CD pipelines; ensures traceability between models, code, and infrastructure

Role	Responsibilities
Documentation & QA Lead	Ensures documentation quality, accessibility, and compliance with SE standards

2.3 Interface Management

Internal Interfaces: - Architecture Infrastructure: SysML models → Terraform/Ansible templates - Infrastructure Integration: Validated deployments → CI/CD artifacts - All Documentation: Model artifacts → Published markdown, diagrams, PDF

External Interfaces: - Stakeholders (executives, operators, developers) - Git repositories (GitHub, GitLab, Azure DevOps) - Cloud platforms (AWS, Azure, GCP, on-premises) - SysML tooling (e.g., Cameo Systems Modeler, Papyrus, open-source alternatives)

---

3. Systems Engineering Processes

3.1 Integrated SE Process Model

The project follows a **model-driven, iterative SE cycle**:

```
Requirements → Architecture → Design → Implementation → Validation → Operations
     ↑                                                                    ↓
                    Feedback & Refinement
```

3.2 Requirements Management

Definition: Capture stakeholder needs and system requirements in SysML requirement blocks.

Key Activities: - Elicit requirements from stakeholders (operators, developers, security, compliance) - Document requirements in SysML format under `sysml/*/` - Establish requirement IDs and traceability links - Manage requirement changes through Git commits with clear commit messages

Artifacts: - `sysml/datacenter/DatacenterRequirements.sysml` - `sysml/infrastructure/InfrastructureRequirements.sysml` - `sysml/overall-design/SysMLCloudPlatfo`

Tools: SysML text/graphical editors, Git version control

3.3 Architecture and Design

Definition: Develop high-level system architecture using SysML block definition diagrams (BDD) and internal block diagrams (IBD).

Key Activities: - Define major system blocks (components, subsystems) - Specify interfaces and connections - Decompose requirements into design elements - Document deployment views and physical architecture

Artifacts: - `sysml/datacenter/DatacenterArchitectureModel.sysml` - `sysml/infrastructure/ClusterArchitectureModel.sysml` - `sysml/overall-design/SysMLCloudPlatform` - `fabrications/*/` — Specific datacenter/rack designs - `configs/*/` — Cluster and infrastructure specifications

Tools: SysML modelers, Graphviz for diagram rendering

3.4 Implementation

Definition: Transform SysML models into executable code, infrastructure definitions, and deployment configurations.

Key Activities: - Generate Terraform modules and Ansible playbooks from SysML deployment views - Create Python/scripting stubs for system components - Define network models, service catalogs, and virtualization configs - Commit implementation artifacts to Git with full traceability

Artifacts: - `library/terraform-modules/` — IaC templates - `library/ansible-roles/` — Deployment automation - Generated scripts and configuration files - Infrastructure-as-Code (IaC) documentation

Tools: Terraform, Ansible, Python, `build-docs.py` script

3.5 Verification and Validation

Definition: Ensure SysML models, requirements, and implementations are correct and complete.

Verification Activities: - Review SysML models for completeness and consistency - Validate model syntax with `scripts/check-encoding.py` - Trace requirements to design elements - Peer review of architecture and design documents

Validation Activities: - Deploy infrastructure using generated Terraform/Ansible - Test system against original requirements - Collect feedback from operators and stakeholders - Validate deployment in staging/production environments

Artifacts: - Test plans and reports - Validation matrices (requirement test case) - Deployment logs and metrics

Tools: Git, SysML tooling, deployment platforms (AWS, Azure, etc.), monitoring tools

3.6 Operations and Maintenance

Definition: Manage the operational system and apply updates/improvements.

Key Activities: - Monitor deployed systems; collect operational metrics - Document design decisions and operational procedures - Process change requests and improvements - Update SysML models to reflect operational feedback

Artifacts: - Operational procedures and runbooks - Incident/change logs - Updated SysML models and deployment configurations

Tools: Monitoring solutions (Prometheus, OpenTelemetry), Git, SysML tooling

---

4. Traceability

4.1 Traceability Links

All SE artifacts maintain bidirectional traceability:

```
Stakeholder Need
    ↓
SysML Requirement (e.g., REQ-001)
    ↓
SysML Design Element (e.g., Block, Interface)
    ↓
Implementation (Terraform, Ansible, Code)
    ↓
Deployment & Operations
    ↓
Feedback (Operational Metrics, Issues)
```

4.2 Traceability Management

- **Requirements**: Stored as SysML `requirement` blocks with `Text`, `Id`, and relationship attributes
- **Design Requirements**: SysML `refines`, `derives`, `satisfies` relationships
- **Implementation Design**: Code comments and configuration metadata reference SysML elements
- **Validation Requirements**: Test cases linked to requirements via Git tags and documentation
- **Tool Support**: Git blame/history, SysML tooling queries, custom traceability dashboards

Repository Structure:

```
sysml/
  overall-design/          # Platform-wide models
  datacenter/              # Datacenter domain
  infrastructure/          # Infrastructure domain
```

```
fabrications/
  fabrication-datacenter-a/
configs/
  infrastructure-cluster-a/
library/
  terraform-modules/
  ansible-roles/
publication/                # Generated traceability reports
```

---

5. Configuration Management

5.1 Baseline Strategy

- **Initial Baseline**: First stable release after architecture review
- **Controlled Baselines**: After each major architecture iteration
- **Development Baseline**: Continuous development on main branch

5.2 Change Control

Change Request Process: 1. Open issue in Git repository (GitHub, GitLab, Azure DevOps) 2. Describe change and impact (model, infrastructure, documentation) 3. Review by Architecture Lead and Integration Engineer 4. Merge branch into main after approval 5. Tag release and update version metadata

Configuration Items: - SysML model files (`.sysml`) - Infrastructure definitions (Terraform, Ansible) - Build scripts and configurations (`Makefile`, `py-requirements.txt`) - Documentation (Markdown, diagrams, PDF)

5.3 Versioning

- Use Git tags for releases: `v1.0.0`, `v1.1.0`, etc. (semantic versioning)
- Maintain version metadata in `Makefile` and model headers
- Document changes in `release-notes/`

---

6. Risk Management

6.1 SE-Specific Risks

Risk	Likelihood	Impact	Mitigation
Model complexity grows unmanageable	Medium	High	Regular architecture reviews, modular design, tool support
SysML tooling changes or becomes obsolete	Low	High	Choose open standards, maintain text-based `.sysml` format, community support
Requirements traceability breaks	Medium	High	Automated traceability checks, Git pre-commit hooks, CI pipeline validation
Deployment validation is incomplete	Medium	High	Comprehensive test matrices, staging environment, monitoring integration
Stakeholder expectations misaligned with model	Medium	Medium	Regular stakeholder reviews, documented assumptions, clear interface definitions

6.2 Risk Monitoring

- Monthly SE metrics review
- Quarterly architecture assessment
- Post-implementation feedback collection

---

7. Data and Information Management

7.1 SE Artifacts

Model Artifacts: - SysML source files (`.sysml`) - Generated diagrams (`.png` from Graphviz) - Data specifications (JSON, YAML)

Documentation: - Markdown pages (architecture, requirements, design decisions) - PDF publications (formal project documentation) - Generated indexes and cross-references

Infrastructure: - Terraform modules and state files - Ansible playbooks and inventory - Deployment logs and metrics

7.2 Repository Structure

```
architecture/
  sysml/                  # SysML source (version controlled)
  publication/            # Generated docs (derived, not committed)
  fabrications/           # Domain-specific designs
  configs/                # Infrastructure configurations
  library/                # Shared modules and utilities
  scripts/                # Build and automation scripts
  templates/              # Documentation and scaffolding templates
```

7.3 Access and Security

- Git repository with role-based access control (RBAC)
- Branch protection rules (main requires approval)
- Audit logging via Git history
- Sensitive data: use environment variables or secrets management (not in repo)

---

8. Metrics and Reporting

8.1 SE Metrics

Metric	Target	Measurement	Frequency
Requirements Traceability (↓→test)	100%	Automated check	Per build
Model Consistency	100%	SysML validation, syntax check	Per commit
Documentation Currency	100%	Auto-generated from models	Per release
Code/Infrastructure Review Coverage	100%	Git pull request checks	Per PR
Deployment Success Rate	>99%	CI/CD pipeline metrics	Monthly

8.2 Reporting

- **Weekly**: SE status update (blockers, progress)
- **Monthly**: Metrics dashboard (traceability, deployment, quality)

- **Quarterly**: Architecture review and risk assessment
- **At Release**: Full traceability matrix and validation report

8.3 Tools and Dashboards

- Git dashboard (commits, branches, PR status)
- Build pipeline dashboard (CI/CD test results)
- Deployment dashboard (infrastructure metrics)
- Documentation index (auto-generated from SysML)

---

9. Tools and Environment

9.1 SE Tools

Tool	Purpose
Git	Version control, branching, CI/CD integration
SysML Modeler	Model creation and editing (Papyrus, Cameo, open-source)
Graphviz	Diagram rendering and visualization
Pandoc + LaTeX	Documentation generation and PDF publication
Python	Build scripts, traceability analysis, custom tools
Terraform	Infrastructure-as-Code (IaC) generation
Ansible	Deployment automation
Monitoring Tools	Prometheus, OpenTelemetry, ELK stack (future)

9.2 Build and Development Environment

Local Development:

```
make init          # Set up .venv and dependencies
make docs          # Build Markdown + diagrams
make pdf           # Generate PDF documentation
make clean         # Clean artifacts
```

CI/CD Pipeline: - Automatic syntax checking (`scripts/check-encoding.py`) - Automatic documentation generation (`scripts/build-docs.py`) - Automated deployment validation - Artifact archiving

9.3 Repository Access

- Primary: GitHub, GitLab, or Azure DevOps

- Branching model: trunk-based or git-flow (team decides)
- Protected branches: `main` requires peer review
- Submodules: Optional for separating large domains

---

10. Training and Competency

10.1 Required Competencies

- **All Team Members:** Git, Markdown, systems thinking
- **Architects:** SysML v2, systems design principles, cloud architecture
- **Infrastructure Engineers:** Terraform, Ansible, cloud platforms, deployment validation
- **Integration Engineers:** CI/CD tools, Python scripting, Git workflows
- **QA/Documentation:** Technical writing, traceability analysis, diagram interpretation

10.2 Training Plan

- **Onboarding:** SysML basics, Git workflows, project structure (1-2 weeks)
- **Ongoing:** Monthly tech talks, quarterly certifications, ad-hoc training on new tools
- **Documentation:** This SEMP, README, CONTRIBUTING guide, inline code comments

10.3 Knowledge Management

- Maintain `CONTRIBUTING.md` for development guidelines
- Keep `README.md` updated with project status
- Document design decisions in SysML comments and git commits
- Archive lessons learned in quarterly reviews

---

11. Interface and Integration Strategy

11.1 SysML-to-Implementation Interfaces

Current Interfaces: - SysML → Markdown: `scripts/build-docs.py` parses `.sysml` and generates documentation - SysML → Diagrams: Graphviz renders block definitions and relationships - Infrastructure models → Terraform: Manual mapping (future: automated generator)

Future Interfaces (Roadmap): - SysML deployment views → Helm chart generator - SysML service specifications → OpenAPI generator - SysML requirements → Test case generator - Operational metrics → Model feedback loop

11.2 Stakeholder Communication

- Architecture reviews: quarterly, all stakeholders
- Change control reviews: per major change
- Status reports: monthly to project leadership
- Incident debriefs: within 24 hours of operational issues

11.3 Documentation & Publication Structure

The project organizes documentation into a three-tier publication structure that aligns with systems engineering disciplines and traceability expectations.

- **System Design (system-level):** a PDF assembled from `overall-design` domain pages. This document describes the high-level architecture that connects all subsystems and provides the principal system-level decisions and interfaces. It is published as `publication/system-design.pdf`.
- **Sub-System Design (subsystem-level):** separate PDF publications for each major subsystem. In this project the primary subsystems are:
  - **Fabrication Subsystem** (datacenter domain) — published as `publication/fabrication-design.pdf` and assembled from `datacenter` domain pages (e.g., `sysml/datacenter/*`).
  - **Infrastructure Subsystem** (infrastructure domain) — published as `publication/infrastructure-design.pdf` and assembled from `infrastructure` domain pages (e.g., `sysml/infrastructure/*`).

  Sub-System Design documents contain the architectural and design rules for their domain and are referenced by technical publications for specific deployments.
- **Technical Publications (deployment-level):** per-site or per-cluster technical publications (for example `fabrications/fabrication-datacenter-a`, `fabrications/fabrication-datacenter-b`, `configs/infrastructure-cluster-a`, `configs/infrastructure-cluster-b`). Each technical publication:
  - Is generated as a distinct PDF (e.g., `publication/fabrications-fabrication-datacenter-a.pdf`
  - References its parent Sub-System Design document (e.g., Fabrication Subsystem) and the System Design where applicable.
  - Contains detailed rack/datacenter or cluster-level models, configuration specs, and deployment instructions.

This organization ensures a clean separation of concerns: - System Design connects the dots and defines global interfaces and constraints. - Sub-System Designs capture domain-specific architecture, constraints, and design patterns. - Technical Publications provide concrete, deployable specifications and instructions tied back to the above designs with full traceability to requirements.

When generating publications via the repository automation (`make docs` and `make pdf`), the pipeline produces one PDF per publication type as described above and writes a `manifest.json` to `publication/` containing the mapping from generated markdown pages to their publication group. This manifest supports downstream processing such as assembling PDFs and generating cross-reference tables.

---

12. Approval and Sign-Off

Approvals and signatures:

- **Systems Engineering Manager** — Plan approval & oversight. Signature: _____ Date: _____
- **Project Manager** — Resource & schedule alignment. Signature: _____ Date: _____
- **Architecture Lead** — Technical feasibility. Signature: _____ Date: _____
- **Stakeholder Representative** — Needs alignment. Signature: _____ Date: _____

---

13. Appendices

A. SysML Domain Breakdown

Overall Design Domain: - Platform-level requirements, interfaces, virtualization model - High-level component architecture

Datacenter Domain: - Datacenter requirements and architecture - Physical layout (rooms, racks, cooling) - Power and cooling interfaces

Infrastructure Domain: - Cluster architecture and networking - Service catalog and deployment models - Virtualization platform specifications

B. Key Documents and Artifacts

- `README.md` — Project overview and quick-start
- `CONTRIBUTING.md` — Development guidelines and submodule workflows
- `publication/` — Generated architecture, requirements, and design documentation
- `.submodules.config` — Submodule configuration for optional domain separation
- `scripts/build-docs.py` — SysML parsing and documentation generation

C. Referenced Standards

- **INCOSE Systems Engineering Handbook** (v4.0+)
- **SysML v2 Specification**
- **Git workflow best practices**
- **Terraform and Ansible best practices**

D. Revision History

- **1.0** — 2025-12-03 — Initial SEMP for SysML Cloud Platform — SE Team

---

Document Classification: Public
Distribution: Project team, stakeholders on request
Next Review: 2026-06-03 (6 months)

ClusterArchitectureModel

Generated: 2025-12-03

Block definition diagram

DatacenterArchitectureModel

Generated: 2025-12-03

Block definition diagram

DatacenterRequirements

Generated: 2025-12-03

Requirement diagram

floorplan

Generated: 2025-12-03

Block definition diagram

Publication index

Generated: 2025-12-03

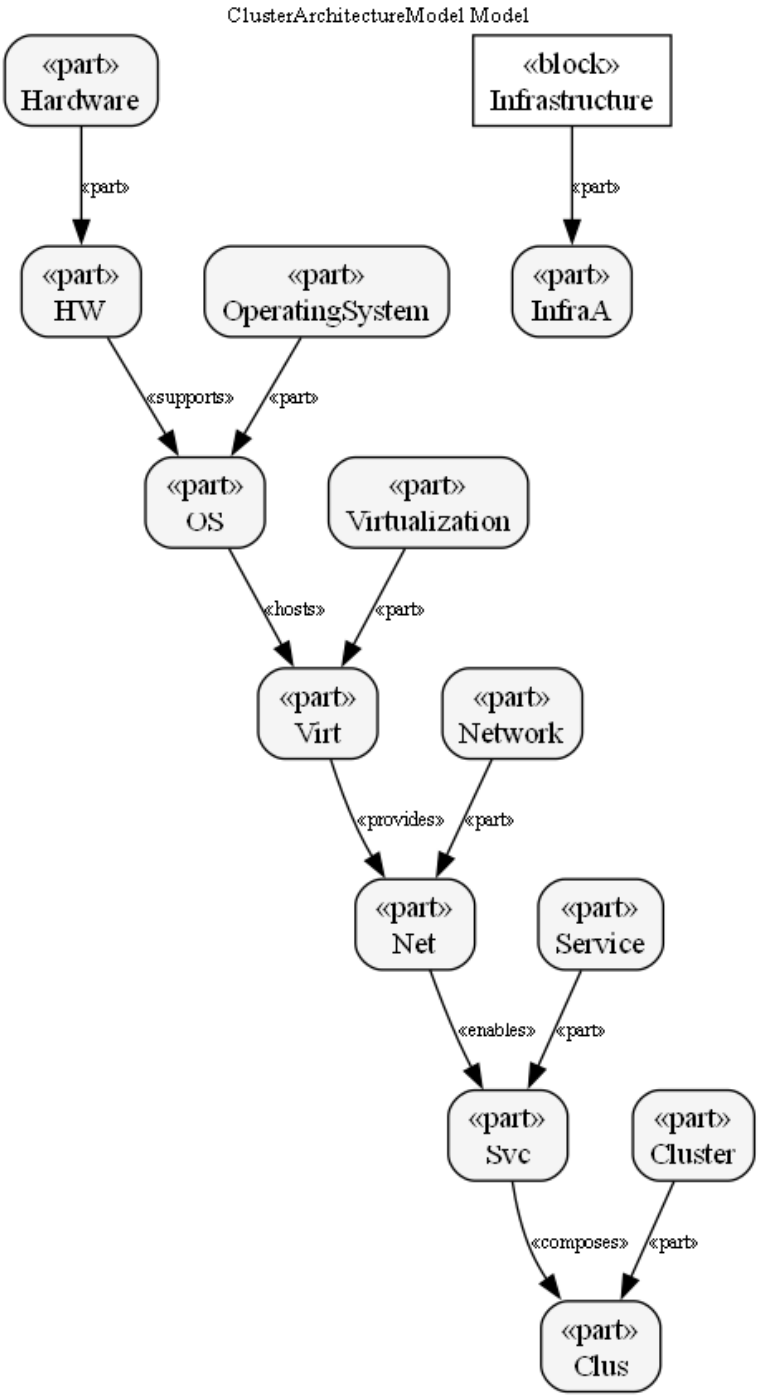

Figure 1: Block definition diagram

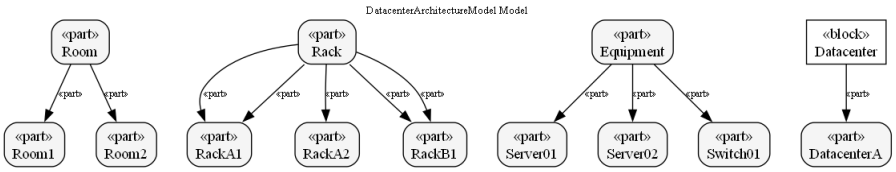

Figure 2: Block definition diagram

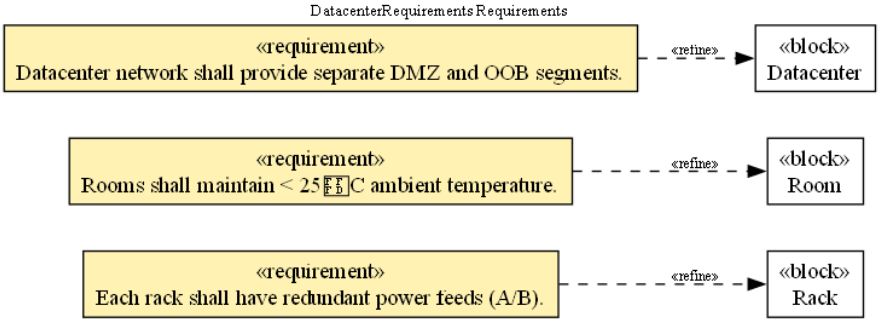

Figure 3: Requirement diagram

floorplan Model

Figure 4: Block definition diagram

Introduction

- Introduction
- Introduction
- Introduction
- floorplan
- floorplan
- rack-models
- rackA1
- rackA1-view
- rackA2
- rackB1
- rackB1-view
- System Engineering Management Plan

Datacenter

- DatacenterArchitectureModel
- DatacenterRequirements
- PowerCoolingInterfaces
- RackModel
- RoomModel

Infrastructure

- ClusterArchitectureModel
- InfrastructureRequirements
- NetworkModel
- ServiceCatalogModel
- VirtualizationModel

InfrastructureRequirements

Generated: 2025-12-03

Requirement diagram

NetworkModel

Generated: 2025-12-03

Block definition diagram

PowerCoolingInterfaces

Generated: 2025-12-03

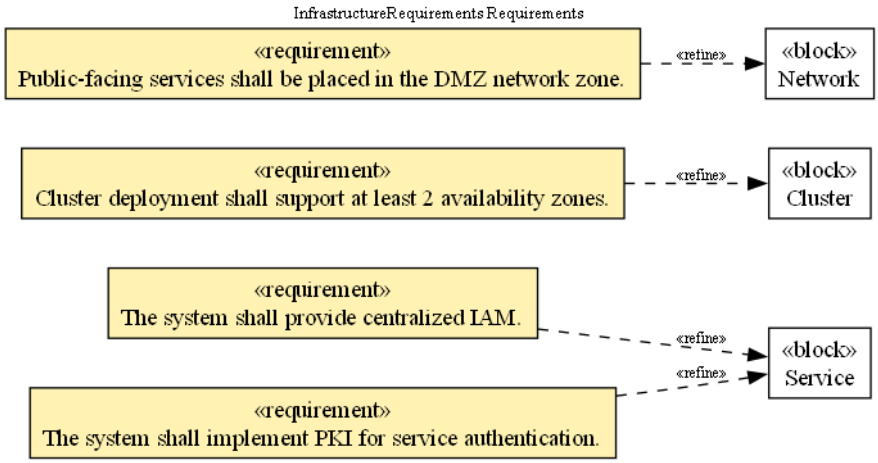

Figure 5: Requirement diagram

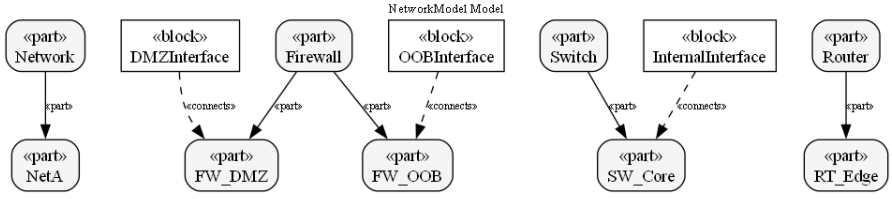

Figure 6: Block definition diagram

Interface diagram

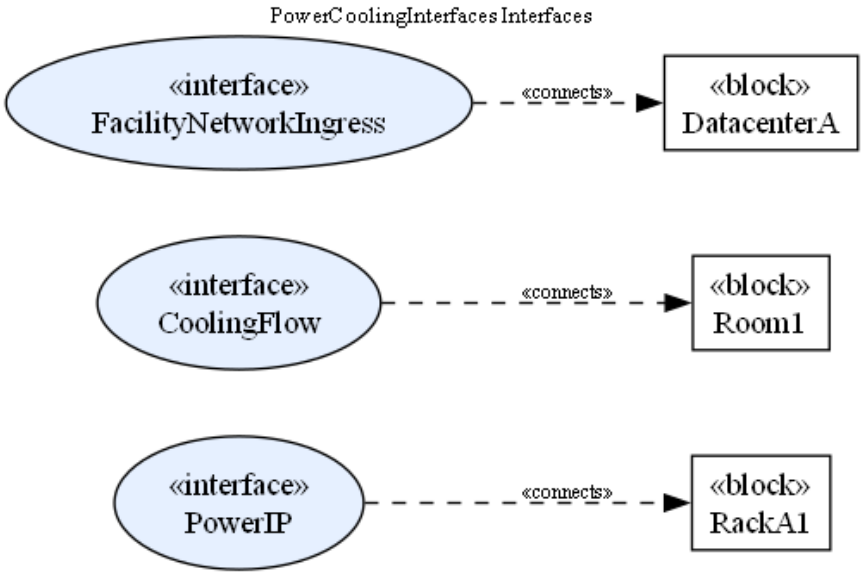

Figure 7: Interface diagram

rackA1-view

Generated: 2025-12-03

Block definition diagram

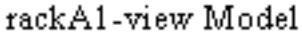

Figure 8: Block definition diagram

rackA1

Generated: 2025-12-03

Block definition diagram

Figure 9: Block definition diagram

rackA2

Generated: 2025-12-03

Block definition diagram

Figure 10: Block definition diagram

rackB1-view

Generated: 2025-12-03

Block definition diagram

Figure 11: Block definition diagram

rackB1

Generated: 2025-12-03

Block definition diagram

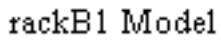

Figure 12: Block definition diagram

RackModel

Generated: 2025-12-03

Block definition diagram

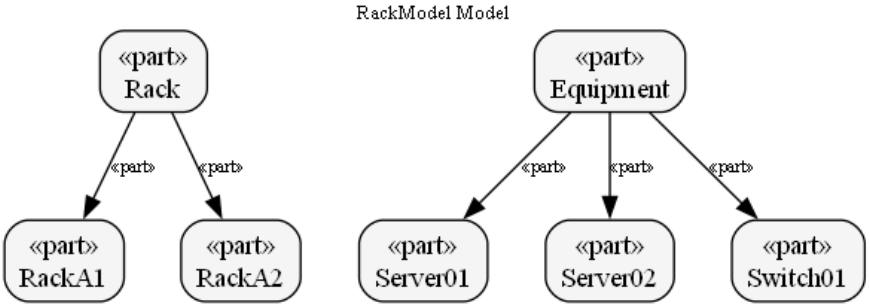

Figure 13: Block definition diagram

rack-models

Generated: 2025-12-03

Block definition diagram

RoomModel

Generated: 2025-12-03

Figure 14: Block definition diagram

Figure 15: Block definition diagram

Block definition diagram

ServiceCatalogModel

Generated: 2025-12-03

Block definition diagram

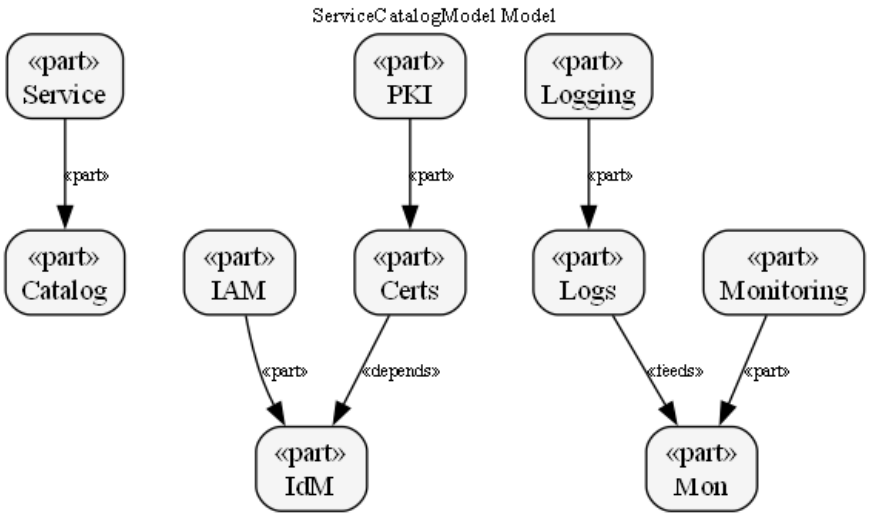

Figure 16: Block definition diagram

Introduction

Generated: 2025-12-03

Interface diagram

Introduction

Generated: 2025-12-03

Block definition diagram

Introduction

Generated: 2025-12-03

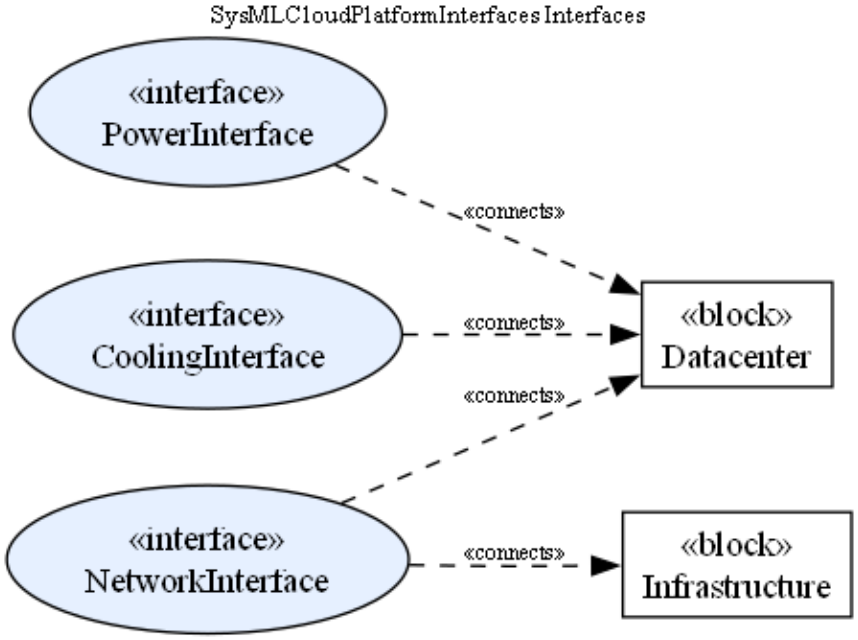

Figure 17: Interface diagram

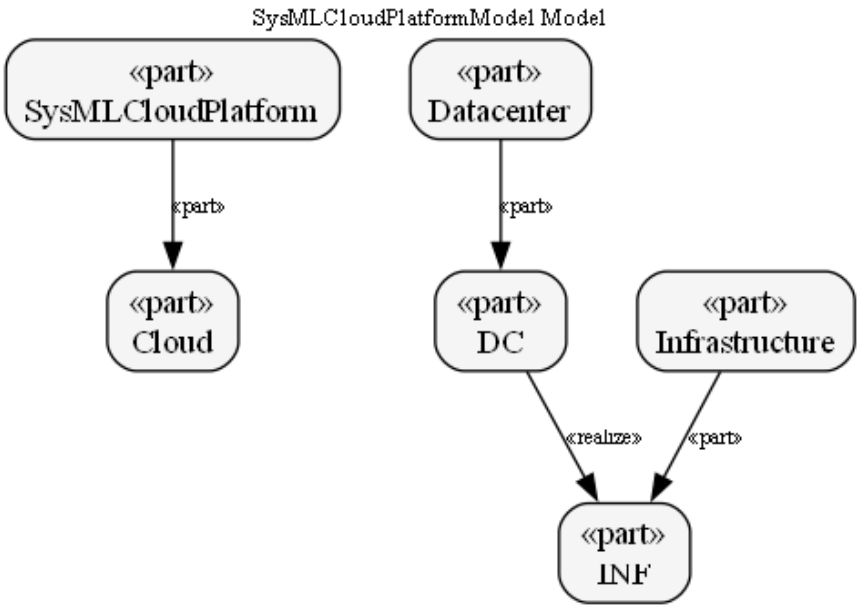

Figure 18: Block definition diagram

Requirement diagram

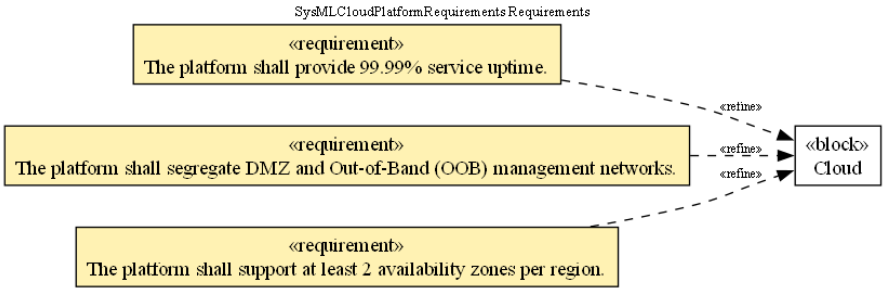

Figure 19: Requirement diagram

VirtualizationModel

Generated: 2025-12-03

Block definition diagram

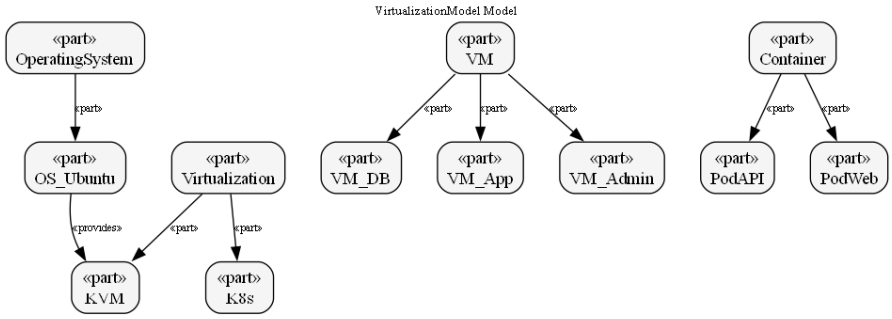

Figure 20: Block definition diagram